Helix Thready — Semantic Search

Table of Contents

Rev	Date	Author	Change
1	2026-07-21	swarm (System Architecture)	Initial draft — Lumen-style in-house search, embeddings, pgvector, rag
2	2026-07-21	swarm (review pass)	Add OpenAPI 3.1 /v1/search contract (§8) per CONVENTIONS §6
3	2026-07-22	swarm (Pass 3 depth)	Close SEM-1 — real `vectordb` `VectorStore` / `Vector` / `SearchQuery` / `SearchResult` / `DistanceMetric` and `embeddings` `EmbeddingProvider` read at source; confirm no `llama` / `hash` provider in `embeddings` (GAP 2.7); deepen search diagram explanation
4	2026-07-22	swarm (docs export)	Fixed inline mermaid syntax so diagram renders

Table of Contents

---

1. Goal & the Lumen model

The original request mandates a “Lumen-style” semantic search — the same “search a codebase by meaning” capability that the Lumen Claude Code plugin provides, but re-implemented **in-house** with llama.cpp/HelixLLM instead of Ollama, and applied to **both** original posts and every generated material `[request §31, research_request_final §15]`. Lumen’s pattern (tree-sitter/AST chunk → `sqlite-vec`) maps onto the in-house stack: `digital.vasic.embeddings` + `digital.vasic.vectordb` (pgvector) + `digital.vasic.rag`, driven by HelixLLM’s OpenAI-compatible `/v1/embeddings`. A fast path exists — point Lumen’s OpenAI-compatible backend at HelixLLM — but the delivered system is the in-house service so it can index posts, transcripts, research docs, OCR text and code uniformly `[research_request_final §15, §18/semantic]`.

2. Subsystem shape (embeddings → vectordb → rag)

The Semantic-search service `[BUILD-NEW, thin]` composes three VERIFIED engines:

Layer	Submodule	Role
Embed	`digital.vasic.embeddings`	Turn text/code chunks into vectors (OpenAI-compat → HelixLLM)
Store/search	`digital.vasic.vectordb` (pgvector)	Cosine `<=>` ANN over vectors, co-located with Postgres
Retrieve/answer	`digital.vasic.rag`	Assemble retrieved context for LLM answers + citations
Tool exposure	`MCP_Module`	Expose search as an MCP tool for CLI agents

Model choices `[research_request_final Q15]`: embeddings `voyage-code-3` / `jina-embeddings-v2-base-code` (code) via HelixLLM `/v1/embeddings`; reranking `BAAI/bge-small-en-v1.5`.

Verified interfaces (SEM-1, read at source this pass). Both engines were read field-by-field:

```
// digital.vasic.vectordb/pkg/client — VERIFIED (Pass 3)
type VectorStore interface {
    Connect(ctx context.Context) error
    Close() error
    Upsert(ctx context.Context, collection string, vectors []Vector) error
    Search(ctx context.Context, collection string, query SearchQuery) ([]SearchResult, error)
    Delete(ctx context.Context, collection string, ids []string) error
    Get(ctx context.Context, collection string, ids []string) ([]Vector, error)
}
type CollectionManager interface { CreateCollection(ctx, CollectionConfig) error;
        DeleteCollection(ctx, name) error; ListCollections(ctx) ([]string, error) }
type Vector       struct { ID string; Values []float32; Metadata map[string]any }
type SearchResult struct { ID string; Score float32; Vector []float32; Metadata map[string]any }
type SearchQuery  struct { Vector []float32; TopK int; Filter map[string]any; MinScore float64 }
type DistanceMetric string // "cosine" | "dot_product" | "euclidean"
type CollectionConfig struct { Name string; Dimension int; Metric DistanceMetric }

// digital.vasic.embeddings/pkg/provider — VERIFIED (Pass 3)
type EmbeddingProvider interface {
    Embed(ctx context.Context, text string) ([]float32, error)
    EmbedBatch(ctx context.Context, texts []string) ([][]float32, error)
    Dimensions() int
    Name() string
}
type Config struct { Model string; BatchSize int; MaxRetries int; Timeout time.Duration }
```

Two facts fall out of the source and shape the design. (1) `vectordb` ships **four** backend clients at source — `pkg/pgvector` (pgxpool; `DistanceOperator(metric)` maps `cosine` → `<=>`), `pkg/qdrant`, `pkg/pinecone`, `pkg/milvus` — all behind the one `VectorStore` seam, so Thready swaps backend by wiring,

not by rewriting callers (this is the mechanism behind `[GAP: 3.1]`: pgvector is the wired/verified MVP backend, the others are present-but-unhardened). (2) `embeddings` ships providers `openai`, `voyage`, `jina`, `google`, `cohere`, `bedrock` — and **no `llama`, `local`, or `hash` provider**. That is the physical confirmation of `[GAP: 2.7]` (no native llama.cpp backend) *and* clarifies `[GAP: 2.1]`: the `HashEmbedder` trap lives in **HelixLLM** (the model server), not in `embeddings`; the local path is `embeddings/pkg/openai` with `BaseURL` pointed at HelixLLM's `/v1/embeddings`.

3. The HashEmbedder trap (must fail loud)

> `[GAP: 2.1]` **(P0) — the single most dangerous trap in the whole stack.** HelixLLM's default local embedder is a **non-semantic `HashEmbedder` stub**: the advertised `all-mpnet-base-v2` is *not actually loaded*; the default `local` / `hash` provider emits deterministic pseudo-vectors with only a startup **WARNING**. Any semantic search built on the default silently returns garbage relevance. VERIFIED at source.

Plan (enforced, not optional):

- Make `HELIX_EMBEDDING_PROVIDER=llama` (llama.cpp `/embedding`) the **enforced default** for any semantic workload; **fail loudly (not warn)** if the hash embedder is selected in a RAG/search context. Load a real local embedding GGUF (code-tuned).
- Parameterize embedding dimension end-to-end (config-driven, validated against the model) — remove HelixLLM's hardcoded 768 (`[GAP: 2.1.2]` RAG dimension hardcoding).
- Confirm `/v1/embeddings` returns the OpenAI `{data:[{embedding,index}]}` shape (consumers sort by `index`); add a contract test (`[GAP: 2.1] improvement`).

```
// Startup guard — refuse to run semantic search on the hash embedder.
func mustSemanticEmbedder(cfg EmbedConfig) (Embedder, error) {
    if cfg.Provider == "" || cfg.Provider == "hash" || cfg.Provider == "local" {
        return nil, fmt.Errorf("REFUSING to start: HELIX_EMBEDDING_PROVIDER=%q is the "+
            "non-semantic HashEmbedder; set =llama with a real embedding GGUF [GAP 2.1]",
        cfg.Provider)
    }
    e := newProvider(cfg)                    // Thready factory → a provider.EmbeddingProvider
    if got := e.Dimensions(); got != cfg.ExpectedDim { // real method is Dimensions() (plural)
        return nil, fmt.Errorf("embedding dim %d != configured %d [GAP 2.1.2]", got,
        cfg.ExpectedDim)
    }
    return e, nil
}
```

This guard is the anti-bluff gate the constitution demands `[CONSTITUTION §11.4.27]`: a green search test must prove real semantic behavior, not a deterministic hash.

4. Embeddings via HelixLLM /v1/embeddings

Embeddings are generated by `digital.vasic.embeddings` pointed at HelixLLM's OpenAI-compatible `/v1/embeddings` endpoint (`HELIX_EMBEDDING_PROVIDER=llama`).

> `[GAP: 2.7]` **No native llama.cpp/Ollama backend package.** `embeddings` local use currently relies on pointing the `pkg/openai` provider's `BaseURL` at an OpenAI-compatible endpoint; `nomic-embed` is not supported. **Plan:** add a first-class **llama.cpp / HelixLLM embeddings provider** (not a repurposed OpenAI client) with health checks + dimension discovery; optionally add a `nomic-embed-code` backend for code fidelity. Until then the OpenAI-compat path is used but is documented as the interim, not the target.

```
// Real embeddings API (VERIFIED): construction is PER-PROVIDER (pkg/openai has its own
// BaseURL/APIKey config); provider.Config carries only {Model,BatchSize,MaxRetries,Timeout}.
// The local path is the openai-compatible provider pointed at HelixLLM (GAP 2.7 interim).
var emb provider.EmbeddingProvider = openai.New(openai.Config{
    BaseURL: helixLLMURL + "/v1",            // …pointed at HelixLLM (GAP 2.7 interim)
    APIKey:  "local",
    Config:  provider.Config{Model: "voyage-code-3", BatchSize: 100, MaxRetries: 3},
})
vecs, err := emb.EmbedBatch(ctx, chunks)      // [][]float32, one per input; Dimensions() must
        match
```

`[GAP: 2.5]` note: HelixLLM defaults `HELIX_LLM_VERIFIER_URL=:7061` while LLMsVerifier serves `:8080`. Reconcile the port in a single config source before wiring the fallback-scoring path; tracked, not assumed working.

5. Chunking strategy

Both **documents and paragraphs** are chunked at symbol/section granularity, not whole files `[research_request_final §19.1]`:

- **Code** (from GitHub research, generated Skills) → **AST / tree-sitter** chunks (function/ symbol level) — the Lumen approach.
- **Docs / research / transcripts / OCR** → Markdown/structured section chunks.
- Each chunk carries `{source_id, kind, span, account_id}` referencing the relational row.

6. pgvector DDL & index

Vectors live in pgvector, co-located in the same Postgres instance as the relational data (keeps the datastore count low, satisfies the < 500 ms SLO) `[research_request_final §2.1.1]`.

```sql
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE semantic_chunks (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    account_id  UUID NOT NULL REFERENCES accounts(id),
    source_id   UUID NOT NULL,          -- FK into posts / assets / research_docs (SoR)
    source_kind TEXT NOT NULL,          -- 'post' | 'reply' | 'transcript' | 'research' | 'ocr'
        | 'code'
    span        JSONB,                  -- {start,end} or {symbol,file}
    content     TEXT NOT NULL,          -- chunk text (redacted surrogate for secrets — see
        security-model)
    embedding   VECTOR(1024) NOT NULL,  -- dim = model dim (voyage-code-3 = 1024); config-
        validated (GAP 2.1.2)
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Cosine ANN index (HNSW for the aggressive < 500 ms SLO); IVFFlat alternative for lower
        memory.
CREATE INDEX idx_chunks_embedding ON semantic_chunks
    USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);
CREATE INDEX idx_chunks_account ON semantic_chunks (account_id, source_kind);

-- Query (cosine distance, tenant-scoped):
--   SELECT source_id, source_kind, content, 1 - (embedding <=> $1) AS score
--   FROM semantic_chunks WHERE account_id = $2
--   ORDER BY embedding <=> $1 LIMIT 20;
```

Forward/rollback migration via `database/pkg/migration.Runner`. Index parameters (`m`, `ef_construction`, `ef_search`) and IVFFlat-vs-HNSW are tuned against the < 500 ms SLO in the testing pack.

7. Relational ↔ semantic relationship

This resolves the progress-doc inconsistency #1 `[research_request_final §2.1.1]`: the **relational store is the system of record**; the vector store holds **embeddings + minimal metadata (source id, kind, span)** that *reference* relational rows. Every vector row carries the relational primary key of its source; semantic search returns ids, which are **hydrated** from the relational store. Because pgvector lives in the same Postgres instance, hydration is a local join, not a cross-datastore call.

8. Query path & the < 500 ms SLO

`[OPERATOR: aggressive SLO — semantic search < 500 ms]`:

1. Embed the query with the **same model** used for indexing (dimension must match — enforced).
2. ANN top-k over pgvector (cosine `<=>`), tenant-scoped by `account_id`.
3. Rerank top-k with `bge-small-en-v1.5`.
4. Hydrate ids from the relational SoR (local join).

5. `rag` assembles context for an answer with citations (for the agent/MCP path); for the `/v1/search` UI path, return ranked hydrated results directly.

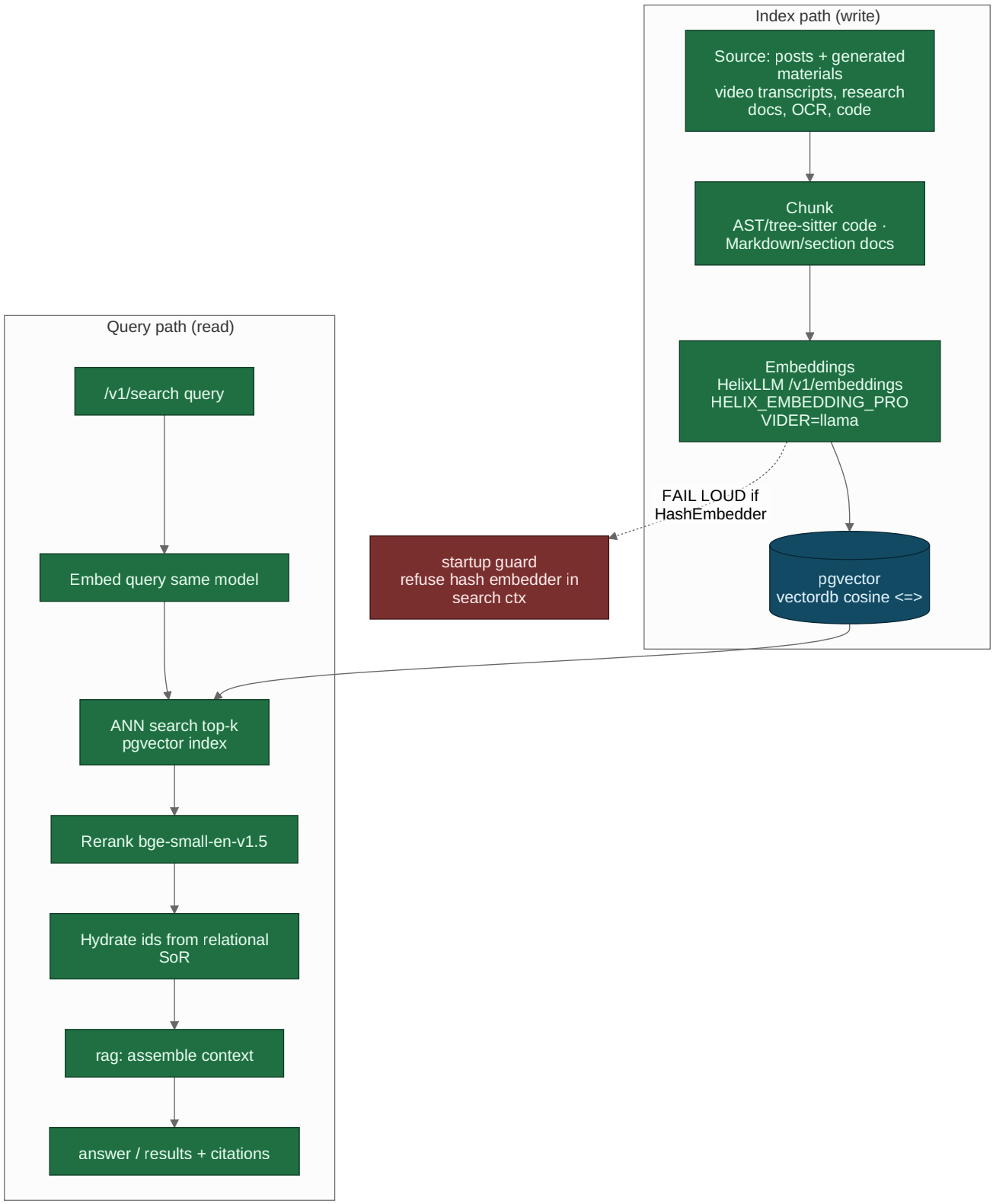
Latency budget: query-embed (cached for repeat queries via `digital.vasic.cache`) + ANN (HNSW, single-digit ms at MVP scale) + rerank + hydrate. The cache tier and HNSW tuning are what hold the < 500 ms line at the 50 TB+ / many-chunk scale.

The `/v1/search` read contract as **OpenAPI 3.1** `[CONVENTIONS §6]` (the full schema set lives in the api/ pack). Results are always tenant-scoped by the caller's `account_id` from the validated token — a query can never reach another account's chunks:

```yaml
openapi: 3.1.0
info: { title: Helix Thready — Semantic search (architecture excerpt), version: "1.0.0" }
paths:
  /v1/search:
    post:
      operationId: semanticSearch
      summary: Meaning-based search over posts and generated materials (< 500 ms SLO).
      security: [ { bearerAuth: [] } ]
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: "#/components/schemas/SearchRequest" }
      responses:
        "200":
          description: Ranked, hydrated results (+ optional rag answer with citations).
          content:
            application/json:
              schema: { $ref: "#/components/schemas/SearchResponse" }
        "401": { description: Missing/invalid credentials }
        "403": { description: RBAC denies the requested account }
        "422": { description: Empty/oversized query }
components:
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }
  schemas:
    SearchRequest:
      type: object
      required: [query]
      properties:
        query:      { type: string, minLength: 1, maxLength: 4096 }
        kinds:      { type: array, items: { type: string,
                      enum: [post, reply, transcript, research, ocr, code] } }
        top_k:      { type: integer, default: 20, minimum: 1, maximum: 100 }
        answer:     { type: boolean, default: false, description: If
        true, rag assembles a cited answer. }
    SearchResponse:
      type: object
      required: [results]
      properties:
        results:
          type: array
          items:
            type: object
            required: [source_id, source_kind, score]
            properties:
              source_id:   { type: string, format: uuid }   # hydrated from the relational SoR
              source_kind: { type: string }
              score:       { type: number, format: float }   # 1 - cosine distance
              snippet:     { type: string }                  # redacted surrogate for sealed
        secrets
        answer:
```

```
      type: object
      nullable: true
      properties:
        text:      { type: string }
        citations: { type: array, items: { type: string, format: uuid } }
```

9. Search diagram



> Rendered PNG/SVG exported via Docs Chain (§11.4.65). Source: `diagrams/semantic-search.mmd`.

Explanation (for readers/models that cannot see the diagram). The subsystem is deliberately mirror-symmetric: a write path that indexes and a read path that queries, joined by the invariant that the *same embedding model* is used on both sides. That symmetry is the whole reason cosine similarity means anything here, so the diagram is best read as two halves that must agree.

On the **write (index) path**, every source — original posts and all generated materials (video transcripts, research docs, OCR text, code) — is chunked (AST/tree-sitter for code, section-wise for docs), embedded through HelixLLM's `/v1/embeddings` with the real llama provider, and written to pgvector via `VectorStore.Upsert(collection, []Vector)` where each `Vector` carries `{ID, Values, Metadata}` and the collection is created with `DistanceMetric = cosine`. A **startup guard** branches off the embed step: if the non-semantic HashEmbedder is configured (a HelixLLM model-server setting, per §2), the service fails loud rather than indexing garbage — this is the enforced closure of `[GAP: 2.1]`, and because the guard runs at boot, a misconfigured deployment never writes a single poisoned vector.

On the **read (query) path**, a `/v1/search` query is embedded with the *same* model, then handed to `VectorStore.Search(collection, SearchQuery{Vector, TopK, Filter, MinScore})` — the `Filter` map is where the `account_id` tenant scope is applied so a query can never reach another account's chunks. The top-k nearest chunks come back as `[]SearchResult{ID, Score, Metadata}`; those are reranked with `bge-small-en-v1.5`, the ids are hydrated from the relational system of record (a local join, because pgvector is co-located), and `rag` assembles the final context so the answer carries citations back to the exact posts/materials.

The pieces the diagram does not draw but the source makes explicit are worth stating: the same `VectorStore` seam admits pgvector today and qdrant/pinecone/milvus by configuration (the `[GAP: 3.1]` swap-by-wiring), and the query-embedding is cache-keyed (`digital.vasic.cache`) so a repeated query skips the model call entirely — one of the levers that holds the < 500 ms SLO. The guard is what stops the whole apparatus from silently degrading to a deterministic hash; the model-symmetry is what makes it correct when it does not.

10. Gap-register coverage

- `[GAP: 2.1]` HashEmbedder stub → enforced `llama` provider + fail-loud guard + dimension validation + `/v1/embeddings` contract test (§3–4).
- `[GAP: 2.7]` Embeddings no native llama.cpp backend → first-class HelixLLM provider planned; OpenAI-compat path documented as interim (§4).
- `[GAP: 3.1]` VectorDB only pgvector wired (Qdrant/Pinecone/Milvus unverified) → **plan:** harden + integration-test **Qdrant** to full pgvector parity (HelixLLM already defaults to Qdrant) so Thready swaps by config; add ANN tuning + benchmark tests for the < 500 ms SLO. pgvector is the MVP backend; others are not claimed working.
- `[GAP: 2.8]` `digital.vasic.memory` search is word-overlap (Jaccard), not semantic → Thready standardizes on the `vectordb` + `embeddings` + `rag` stack, **not** `Memory`, for real recall; `HelixMemory` (4 external services) deferred.

- `[GAP: 2.5]` HelixLLM verifier port `:7061` vs `:8080` → reconcile in single config source (§4).

11. TDD reproduce-first skeletons

```
// RED: semantic search must NOT run on the hash embedder.
func TestEmbedder_RefusesHash(t *testing.T) {
    _, err := mustSemanticEmbedder(EmbedConfig{Provider: "hash"})
    require.Error(t, err) // fail loud, not warn (GAP 2.1)
}

// RED: index and query dims must match.
func TestEmbed_DimensionEnforced(t *testing.T) {
    _, err := mustSemanticEmbedder(EmbedConfig{Provider:"llama", Model:"voyage-code-3",
        ExpectedDim: 999})
    require.Error(t, err) // model dim 1024 != 999 (GAP 2.1.2)
}

// RED: /v1/embeddings must return OpenAI shape ordered by index.
func TestEmbeddings_OpenAIShape(t *testing.T) {
    out := callEmbeddings(t, []string{"a","b"})
    require.Equal(t, []int{0,1}, indices(out.Data)) // consumers sort by index
}

// RED: search results are tenant-scoped (no cross-account leakage).
func TestSearch_TenantScoped(t *testing.T) {
    seed(t, account="A", "secret-a"); seed(t, account="B", "secret-b")
    res := search(t, account="A", "secret")
    require.NotContains(t, sourceAccounts(res), "B")
}
```

12. Open items

- `[CLOSED: SEM-1]` (was: `vectordb` / `embeddings` interfaces unread). **Source-verified this pass**: `vectordb/pkg/client.VectorStore` (`Connect` / `Close` / `Upsert` / `Search` / `Delete` / `Get`) + `Vector` / `SearchQuery{Vector,TopK,Filter,MinScore}` / `SearchResult{ID,Score,Metadata}` / `DistanceMetric` / `CollectionConfig`, with backend clients `pgvector` / `qdrant` / `pinecone` / `milvus`; and `embeddings/pkg/provider.EmbeddingProvider` (`Embed` / `EmbedBatch` / `Dimensions` / `Name`) + `Config{Model,BatchSize,MaxRetries,Timeout}` with providers `openai/voyage/jina/google/cohere/bedrock` (no `llama` / `hash`). §2/§4 now use the real surfaces; the pgvector **DDL** in §6 is Thready-side migration (the `pgvector` client operates over an existing table+index), reconciled and consistent.
- `[OPEN: SEM-2]` Real code-tuned embedding GGUF selection (voyage-code-3 is a hosted model; the *local* equivalent GGUF, e.g. `nomic-embed-code`, and its dimension) needs a benchmark decision `[RESEARCH]`; tracked as a P0 workable item alongside the HelixLLM embedder fix.

- `[OPEN: SEM-3]` HNSW vs IVFFlat and `ef_search` tuning against the < 500 ms SLO at full scale is deferred to the testing/benchmark pack.

Made with love ♥ by Helix Development.